

计算导论 小组作业

技术报告

电院 2400 A 524030910000

B 524030910000

C 524030910000

2025.1.2

整体目标：实现一个 Matrix 类。

说明：部分函数首行的 `assert isinstance(other, Matrix)` 是为了程序编写时 IDE 的自动提示。

```
# Framework for IEEE course final project
# Fan Cheng, 2022

import random
from typing import Iterable

class Matrix:
    data: list[list]
    dim: tuple[int, int]

    def __init__(self, data=None, dim=None, init_value=0):
        if data != None:
            assert type(data) == list and len(data) > 0
            self.data = data
            self.dim = len(data), len(data[0])
        elif dim != None:
            assert type(dim) == tuple
            self.data = [[init_value] * dim[1] for _ in range(dim[0])]
            self.dim = dim
        else:
            raise Exception("data和dim不能同时为None")
```

定义类属性，初始化数据。若 `data` 非空，则根据 `data` 初始化 `dim`，否则若 `dim` 非空，根据 `init_value` 生成矩阵，否则抛出异常。

```
def shape(self) -> tuple[int, int]:
    return self.dim
```

直接返回 `dim`。

```
def reshape(self, newdim):
    m1, n1 = newdim
    if m1 * n1 != self.dim[0] * self.dim[1]:
        raise Exception("元素个数不符，无法拉伸")
    flat = []
    for row in self.data:
        flat += row
    matrix = [flat[n1 * m: n1 * (m + 1)] for m in range(m1)]
    return Matrix(matrix)
```

矩阵拉伸。先将所有元素摊平到一个 `list`，再按 `newdim` 拼接。若元素个数不对应，抛出异常。

```
def dot(self, other):
    assert isinstance(other, Matrix)
    assert self.dim[1] == other.dim[0], "行列数不匹配，无法相乘"
    ans = Matrix(dim=(self.dim[0], other.dim[1]))
    for i in range(ans.dim[0]):
        for j in range(ans.dim[1]):
            for k in range(self.dim[1]):
                ans[i, j] += self[i, k] * other[k, j]
    return ans
```

先创建一个空矩阵，再按矩阵乘法公式填充数据。若行列数不匹配，抛出异常。

```
def T(self):
    res = Matrix(dim=(self.dim[1], self.dim[0]), init_value=0)
    for i in range(self.dim[1]):
        for j in range(self.dim[0]):
            res[i, j] = self[j, i]
    return res
```

先创建一个空矩阵，再按公式填充数据。（索引函数见后）

```
def sum(self, axis=None):
    if axis == 0:
        res = [0] * self.dim[1]
        for i in range(self.dim[0]):
            for j in range(self.dim[1]):
                res[j] += self.data[i][j]
        return Matrix(list(map(lambda x: [x], res)))
    elif axis == 1:
        return Matrix([[sum(_) for _ in self.data]])
    elif axis == None:
        return Matrix([[sum(sum(_) for _ in self.data)])])
    else:
        raise Exception
```

矩阵元素求和。按列求和则维护一个每列之和的数组，再用匿名函数和 `map` 将每个数转换成单元素数组，最后转换成 `list`。按行求和则对每行使用内置函数 `sum`。全部求和则嵌套两层 `sum`。

```
def copy(self):
    return Matrix([row[:] for row in self.data])
```

深拷贝。利用列表索引复制每行，并用循环拼接。

```
def Kronecker_product(self, other):
    assert isinstance(other, Matrix)
    ans = Matrix(dim=(self.dim[0] * other.dim[0],
                      self.dim[1] * other.dim[1]))
    for i in range(other.dim[0]):
        for j in range(other.dim[1]):
            ans[i * self.dim[0]: (i + 1) * self.dim[0], j * self.dim[1]: (
                j + 1) * self.dim[1]] = self * Matrix(dim=other.dim, init_value=other[i, j])
    return ans
```

借助矩阵切片赋值（实现见后），先创建空矩阵，再按公式实现 Kronecker 积。

```
def __getitem__(self, key):
    k1, k2 = key
    if type(k1) == int and type(k2) == int:
        return self.data[k1][k2]
    elif isinstance(k1, slice) and isinstance(k2, slice):
        k1 = slice(0 if k1.start is None else k1.start,
                    self.dim[0] if k1.stop is None else k1.stop,
                    k1.step)
        k2 = slice(0 if k2.start is None else k2.start,
                    self.dim[1] if k2.stop is None else k2.stop,
                    k2.step)
        return Matrix([[self.data[i][j] for j in range(k2.start, k2.stop)] for i in
                        range(k1.start, k1.stop)])
    else:
        raise Exception

def __setitem__(self, key, value):
    k1, k2 = key
    if (isinstance(k1, int) or isinstance(k1, float)) and (isinstance(k2, int) or
                                                              isinstance(k2, float)) and (isinstance(value, int) or isinstance(value, float)):
        self.data[k1][k2] = value
    elif isinstance(k1, slice) and isinstance(k2, slice) and isinstance(value, Matrix):
        k1 = slice(0 if k1.start is None else k1.start,
                    self.dim[0] if k1.stop is None else k1.stop,
                    k1.step)
        k2 = slice(0 if k2.start is None else k2.start,
                    self.dim[1] if k2.stop is None else k2.stop,
                    k2.step)
        assert value.dim == (k1.stop - k1.start, k2.stop - k2.start)
        for i in range(k1.start, k1.stop):
            for j in range(k2.start, k2.stop):
                self.data[i][j] = value.data[i - k1.start][j - k2.start]
    else:
```

```
raise Exception("值类型错误")
```

索引和赋值函数。先对 `key` 进行解包，对数据类型进行校验。单元素索引和赋值实现简单。对于切片索引和赋值，将传入的 `slice` 类型进行调整，将 `start` 和 `stop` 为 `None` 时分别调整为 0 和某一维度，再返回对应位置的数据（索引），或循环将数据设为输入矩阵对应位置的数据（赋值）。

```
def __pow__(self, n):
    if self.dim[0] != self.dim[1]:
        raise Exception("不是方阵，无法求幂")
    if n == 0:
        return I(self.dim[0])
    matrix = self.copy()
    for _ in range(n - 1):
        matrix = matrix.dot(self)
    return matrix

def __add__(self, other):
    if self.dim[0] != other.dim[0] or self.dim[1] != other.dim[1]:
        raise Exception("两个矩阵行数与列数不一致，无法相加")
    matrix = Matrix(dim=self.dim)
    for i in range(self.dim[0]):
        for j in range(self.dim[1]):
            matrix.data[i][j] = self.data[i][j] + other.data[i][j]
    return matrix

def __sub__(self, other):
    if self.dim[0] != other.dim[0] or self.dim[1] != other.dim[1]:
        raise Exception("两个矩阵行数与列数不一致，无法相减")
    matrix = Matrix(dim=self.dim)
    for i in range(self.dim[0]):
        for j in range(self.dim[1]):
            matrix.data[i][j] = self.data[i][j] - other.data[i][j]
    return matrix

def __mul__(self, other):
    if self.dim[0] != other.dim[0] or self.dim[1] != other.dim[1]:
        raise Exception("两个矩阵行数与列数不一致，无法对应位置相乘")
    matrix = Matrix(dim=self.dim)
    for i in range(self.dim[0]):
        for j in range(self.dim[1]):
            matrix.data[i][j] = self.data[i][j] * other.data[i][j]
    return matrix

def __len__(self):
    return self.dim[0] * self.dim[1]
```

幂、加法、减法、乘法、元素个数函数。实现方式简单。

```
def __str__(self):
    print_value = []
```

```

line = [0] * self.dim[1]
for i in range(self.dim[0]):
    for j in range(self.dim[1]):
        len1 = len(str(self.data[i][j]))
        if len1 > line[j]:
            line[j] = len1
for i in range(self.dim[0]):
    if i == 0:
        print_value.append("[[")
    else:
        print_value.append(" [")
    for j in range(self.dim[1]):
        len1 = len(str(self.data[i][j]))
        if j == 0:
            a = [" "] * (line[j] - len1)
        else:
            a = [" "] * (line[j] + 1 - len1)
        print_value.extend(a)
        print_value.append(str(self.data[i][j]))
    if i == self.dim[0] - 1:
        print_value.append("]]")
    else:
        print_value.append("]\n")
return "".join(print_value)

```

先维护每列元素长度的最大值 `line: list[int]`。对于每个数，在前方补齐空格（首行少补一个），保证一列的右侧对齐。在每行首位加上中括号（首/末行两个左/右括号）。

```

def det(self):
    if self.dim[0] != self.dim[1]:
        raise Exception("该矩阵非方阵，无法求行列式")
    matrix = self.copy()
    times = 0
    for i in range(matrix.dim[1]):
        for j in range(i, matrix.dim[0]):
            if matrix.data[j][i] != 0:
                if i != j:
                    matrix.data[i], matrix.data[j] = matrix.data[j], matrix.data[i]
                    times += 1
                break
        if matrix.data[i][i] == 0:
            return 0
        for j in range(i + 1, matrix.dim[0]):
            k = matrix.data[j][i] / matrix.data[i][i]
            for n in range(matrix.dim[1]):
                matrix.data[j][n] -= k * matrix.data[i][n]
    det_value = 1
    for i in range(matrix.dim[0]):
        det_value *= matrix.data[i][i]
    if times % 2 == 1:

```

```
    det_value *= -1
    return det_value
```

求行列式函数。首先检查矩阵的行数是否等于列数，若不是方阵抛出异常，若是方阵则进行高斯消元。高斯消元的过程如下：用 `times` 记录交换行的次数，然后从矩阵的第一列开始，找到第一个非零元素（主元）。如果主元不在第一行，则通过行交换将主元移到第一行。对于主元下方的每一行，通过行变换将主元下方的元素变为零。通过该过程得到上三角矩阵，然后计算对角线上元素的乘积并乘上 $(-1)^{\text{times}}$ 得到行列式的值。

```
def inverse(self):
    if self.dim[0] != self.dim[1]:
        raise Exception("该矩阵非方阵，无法求逆矩阵")
    matrix = self.copy()
    inverse_matrix = Matrix(dim=self.dim)
    for i in range(self.dim[0]):
        inverse_matrix.data[i][i] = 1 # 在右侧加入一个单位矩阵，变化后的结果即为逆矩阵
    for i in range(matrix.dim[0]):
        if matrix.data[i][i] == 0:
            raise Exception("该阵为奇异阵")
        for j in range(matrix.dim[0]):
            if j != i:
                k = matrix.data[j][i] / matrix.data[i][i]
                for n in range(matrix.dim[1]):
                    matrix.data[j][n] -= k * matrix.data[i][n]
                    inverse_matrix.data[j][n] -= k * inverse_matrix.data[i][n] # 同步操作
    for i in range(matrix.dim[0]):
        k = 1 / matrix.data[i][i]
        for n in range(matrix.dim[1]):
            inverse_matrix.data[i][n] *= k
    return inverse_matrix
```

求逆矩阵函数。首先检查矩阵是否为方阵。若不是方阵抛出异常，若是方阵则通过求逆矩阵的初等变换法求逆矩阵。创建一个与原矩阵同维度的单位矩阵，将原矩阵转换为上三角矩阵（同上）（若在消元过程中主对角线元素为零则抛出异常），再将上三角矩阵转换为单位矩阵，在此过程中对单位矩阵进行同步操作，原单位矩阵同步的结果即为所求逆矩阵。

```
def rank(self):
    matrix = self.copy()
    for i in range(matrix.dim[1]):
        x = None
        for j in range(i, matrix.dim[0]):
            if matrix.data[j][i] != 0:
                x = j
                break
        if x is None:
            continue
        if i != x:
            matrix.data[i], matrix.data[x] = matrix.data[x], matrix.data[i]
        for j in range(i + 1, matrix.dim[0]):
```

```

        k = matrix.data[j][i] / matrix.data[i][i]
        for n in range(matrix.dim[1]):
            matrix.data[j][n] -= k * matrix.data[i][n]
    rank = 0
    for line in matrix.data:
        for a in line:
            if a != 0:
                rank += 1
                break
    return rank

```

求矩阵秩函数。将原矩阵化为行阶梯形矩阵（过程同化方阵为上三角矩阵），然后通过遍历行阶梯形矩阵的每一行，统计非零行的数量，从而得到矩阵的秩。

```

def I(n):
    matrix = Matrix(dim=(n, n))
    for i in range(n):
        matrix.data[i][i] = 1
    return matrix

def narray(dim, init_value=1):
    return Matrix(dim=dim, init_value=init_value)

def arange(start, end, step=1):
    n = (end - start) // step
    data = [[start + i * step for i in range(n)]]
    return Matrix(data)

def zeros(dim):
    return narray(dim, init_value=0)

def zeros_like(matrix: Matrix):
    return narray(matrix.dim, init_value=0)

def ones(dim):
    return narray(dim, init_value=1)

def ones_like(matrix: Matrix):
    return narray(matrix.dim, init_value=1)

def nrandom(dim):
    data = [[random.random() for __ in range(dim[0])]
             for _ in range(dim[1])]
    return Matrix(data)

def nrandom_like(matrix: Matrix):
    data = [[random.random() for __ in range(matrix.dim[0])]
             for _ in range(matrix.dim[1])]
    return Matrix(data)

```

实现方式简单。

```
def concatenate(items: Iterable[Matrix], axis=0):
    assert items, "items不能为空"
    first_matrix = items[0]
    assert isinstance(first_matrix, Matrix), "矩阵类型不对"
    assert axis in [0, 1], "axis必须为0或1"
    first_shape = first_matrix.shape()
    if axis == 0:
        columns = first_shape[1]
        for item in items:
            if item.shape()[1] != columns:
                raise Exception(
                    "矩阵的列数不相等")
    else:
        rows = first_shape[0]
        for item in items:
            if item.shape()[0] != rows:
                raise Exception(
                    "矩阵的行数不相等")
    result_data = []
    if axis == 0:
        result_data = [row for matrix in items for row in matrix.data]
    else:
        result_data = []
        for row in range(rows):
            result_row = []
            for item in items:
                result_row.extend(item.data[row])
            result_data.append(result_row)
    return Matrix(result_data)
```

拼接多个矩阵。先进行数据校验，再按行或列进行拼接。

```
def vectorize(func):
    def f(matrix: Matrix) -> Matrix:
        new_matrix = matrix.copy()
        for i in range(new_matrix.dim[0]):
            for j in range(new_matrix.dim[1]):
                new_matrix[i, j] = func(new_matrix[i, j])
        return new_matrix
    return f
```

实现一个装饰器（decorator）。在函数内部嵌套一个函数，将传入的函数 `func` 作用在每个数上。最后返回这个函数。